

Anchoring Critic-Guided Fine-Tuning of Generative Robot Policies

Anonymous Authors

Abstract—Reinforcement learning can improve pretrained robot policies, but imperfect critic guidance can also disrupt the useful behaviors they have already learned. Even with a frozen base policy, repeated critic-guided corrections can gradually override useful pretrained behavior, making small updates alone insufficient to ensure its preservation. We present CAST (Constrained Action-Space Transport), a likelihood-free method that learns residual corrections to a frozen generative policy. CAST constructs action targets by combining critic guidance with a soft pull toward the corresponding base actions and capping each proposed correction. The residual then learns these targets by regression. This design maintains a persistent reference to pretrained behavior while leaving room for adaptation. Experiments in simulated manipulation show improved task success and fewer environment steps needed to reach a target success rate. Real-robot experiments on OpenArm further show improved placement and stacking success over the pretrained policies. Ablations show that restoration can preserve task success where clipping alone fails, supporting the importance of a persistent reference during refinement. Broader evaluations reveal that the benefits of anchoring vary across tasks, highlighting the need to balance preserving useful behavior with allowing the changes required for improvement.

I. INTRODUCTION

A robot may execute most actions in a task correctly yet still fail because of a single critical error. For example, it may successfully approach, grasp, and position an object but fail at the final release. Under sparse task-level rewards, such a near-success receives no credit for the useful actions preceding the failure. Refinement should therefore improve the failed execution without disrupting the behavior that already works.

Generative policies pretrained on demonstrations provide useful behavior that reinforcement learning (RL) can refine through interaction and task feedback [1], [2]. Residual RL implements this refinement by adding learned action corrections to a frozen base policy [3]. Critic-guided refinement, however, requires both accurate value estimates and action gradients that point toward useful corrections. A critic may recognize failing behavior yet provide an unreliable improvement direction. With sparse rewards and limited experience, following such gradients can exploit critic errors and degrade actual task performance [4], [5]. Freezing the base does not remove this risk: small residual updates can accumulate and eventually override its behavior, as illustrated in Fig. 1.

To limit this accumulated drift, we introduce CAST (Constrained Action-Space Transport), a likelihood-free method that converts critic guidance into constrained targets for residual learning. Building on action-target training [6], [7] within DICE-RL’s frozen-base residual architecture, CAST pairs each current action with the corresponding base action generated from the same noise sample. The critic proposes a

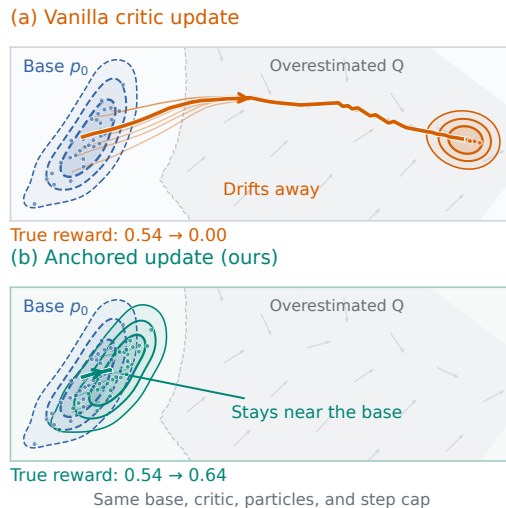


Fig. 1: Preserving pretrained robot skills during refinement. In robot manipulation, refinement should correct placement or release errors while preserving useful reaching and grasping behavior. This synthetic action-space example illustrates how critic-guided updates can disrupt pretrained actions. Both panels share a frozen critic, paired initial actions from the blue base distribution, and a per-update step cap. (a) Critic ascent moves actions into an overestimated region (gray), reducing true reward. (b) Projection around each initial action preserves a fixed base reference and improves reward. Curves show representative updates; contours enclose 50%, 80%, and 95% of estimated action probability.

normalized and clipped local correction, while a soft restoring field opposes accumulated departure from the paired base action. The combined displacement is then capped, and the residual is trained toward the resulting detached target. Because the frozen generator supplies only samples, the update requires neither policy likelihoods nor gradients through the base. The restoring reference remains soft, moderating critic guidance without assuming that each base action is correct or forcing the target to remain within the reference radius.

Across matched simulations, CAST improves both task success and interaction efficiency over standard residual refinement, achieving a 9.4-percentage-point mean gain across eight LIBERO-90 tasks and requiring approximately 38% fewer environment steps to reach 90% success on robomimic square. A five-task component study further supports restoration as an effective mechanism for preserving useful pretrained behavior during refinement. Real-robot evaluations confirm these gains, improving placement and stacking success by 40 and 35 percentage points, respectively.

Our contributions are:

- CAST, a likelihood-free residual-refinement framework that converts critic guidance into explicit action targets

using only samples from a frozen generative policy.

- A same-noise anchoring mechanism that assigns each candidate a persistent base reference and combines radial restoration against accumulated drift with displacement clipping of each requested correction.
- Analyses connecting constrained target regression to regularized actor learning and same-noise residual coupling to an upper bound on the squared 2-Wasserstein distance between candidate action distributions.

II. RELATED WORK

A. Fine-Tuning and Steering Generative Policies

Reward feedback can enter a generative policy through its training objective or its sampling procedure. DPPO optimizes the diffusion denoising process [1]; ReinFlow introduces stochastic flow transitions with tractable likelihoods [2]; and Flow Policy Optimization uses a flow-matching loss surrogate [8]. Diffusion-QL combines diffusion behavior cloning with value maximization [9], while Flow Q-Learning trains a one-step actor with critic guidance and distillation regularization [10].

A frozen policy can also be improved by changing how its samples are obtained or selected. DSRL learns the input noise distribution [11], and V-GPS ranks sampled actions using an offline-trained value function [12]. IBRL uses imitation-policy proposals for exploration and value bootstrapping [13]. CAST operates on the generator’s output: it learns residual action corrections while retaining the frozen base’s sampling procedure.

B. Critic-Guided Action and Particle Updates

Critic guidance can shape action targets or the objective used to train a generative policy. DIPO improves stored actions by critic-gradient ascent [6], and PA-RL combines candidate ranking and local action optimization with supervised fitting [14]. Q-Score Matching (QSM) instead relates the policy score to the critic’s action gradient [7]. AWR, MPO, and IDQL also use weighted fitting or action selection [15]–[17].

Particle updates connect CAST to Soft Q-Learning’s amortized Stein sampler [18] and drifting models’ displaced-sample regression [19]. Drifting Field Policy combines top- k critic-selected targets with drift losses that anchor behavior [20]. CAST uses fresh residual targets to study how improvement and preservation should be combined. Its main variant pairs each candidate with its same-noise base output, adds radial restoration to critic guidance, and clips the requested correction. Value-only variants examine alternatives to the local action gradient.

C. Residual Refinement and Behavioral Constraints

Residual RL augments existing controllers with learned corrections [21]. ResFiT extends this approach to frozen action-chunked BC policies under sparse rewards [22]. Policy Decorator bounds residual outputs and progressively introduces exploration [23]; DAWN addresses critic warm-up and scale mismatch [24]. DICE-RL combines residual value maximization, selective behavior regularization, and value-

guided action selection [3].

Behavioral constraints also limit exploitation of critic error in offline RL. BEAR restricts actions toward dataset support [25], while TD3+BC adds a cloning penalty to the actor objective [26]. Fisher Decorator constrains residual transport using Fisher geometry estimated from flow-policy scores [27]. CAST applies radial restoration and per-candidate displacement caps to explicit action targets, using only samples from the base. Its restoring terms depend on departure from the base, independently of the critic’s preference for the current action. Within DICE-RL, we test whether clipping alone can replace this reference and compare the resulting update with loss-space penalties and hard target projection.

III. PROBLEM STATEMENT

A. Setting

We study off-policy reinforcement learning to refine a pretrained generative policy π_0 while retaining useful pretrained behavior. Given an observation s and noise z , the policy produces an action chunk $a \in \mathbb{R}^{Hd}$ consisting of H controls of dimension d . Following prior work on residual RL [3], [21], [22], we keep π_0 frozen and learn an additive correction r_θ :

$$a_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z), \quad z \sim \mathcal{N}(0, I). \quad (1)$$

An ensemble critic learns expected returns through temporal-difference updates on replayed transitions from demonstrations and policy rollouts. Using this critic, CAST constructs action targets and trains the residual to match them by regression.

B. Accumulated Departure from the Base

An imperfect critic may overestimate the value of actions poorly represented in its training data, causing repeated critic-guided updates to move the policy away from useful pretrained behavior [5]. Limiting each correction does not prevent this accumulated drift: for a fixed observation and noise sample, $|a_{t+1} - a_t|_2 \leq \delta$ bounds each update, but the distance from the initial action can still grow over optimization steps t . Fig. 1 illustrates this behavior under a fixed critic: unanchored ascent moves actions toward an overestimated region, whereas fixed-base projection keeps them near their starting points. This motivates a persistent restoring pull toward the base together with a cap on each proposed correction, preserving useful behavior while allowing adaptation. Because these constraints shape the actor’s training targets, they do not guarantee that the fitted policy outputs satisfy the same bounds.

IV. METHOD

CAST retains DICE-RL’s off-policy residual framework [3]: the pretrained policy $\pi_0(s, z)$ is frozen, while the residual actor $r_\theta(s, z)$ and critic ensemble $Q_\phi(s, z, a)$ are trained. Here, s is the observation, z the noise, and a the action chunk; noise-independent critics omit z . From replayed demonstrations and policy rollouts, the critic learns expected discounted returns using temporal-difference targets that combine rewards with predictions from a slowly updated

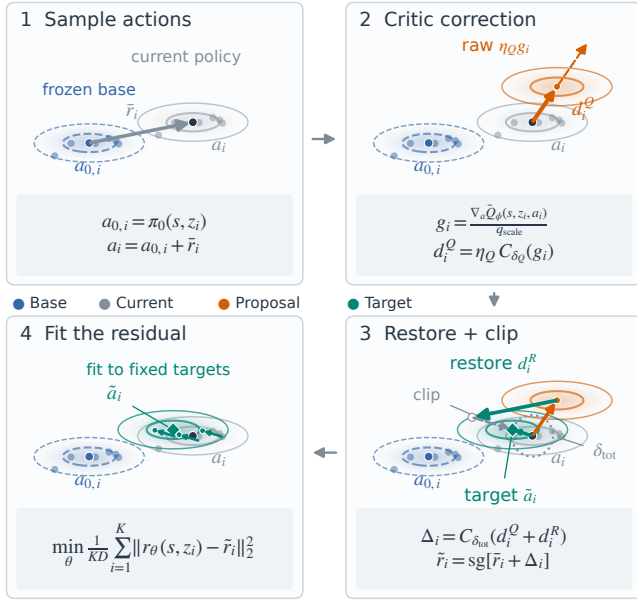


Fig. 2: CAST builds fixed targets to refine robot actions. At a fixed observation s , read clockwise: (1) sample paired base and current actions using the same noise z_i ; (2) propose a clipped critic correction; (3) add restoration toward the base and clip the combined step; (4) regress the residual onto the fixed targets. In step 2, the dashed orange ray extends the uncapped request $\eta_Q g_i$. In step 3, restoration is evaluated at the current action and drawn head-to-tail with the critic correction; the dotted circle bounds the target step by δ_{tot} . Here $\tilde{r}_i = \text{sg}[r_\theta(s, z_i)]$ and $\tilde{a}_i = a_{0,i} + \tilde{r}_i$, where sg stops gradients. All panels share a coordinate scale; green shows the targets before actor fitting.

target critic. Our main variant replaces DICE-RL’s actor objective with regression to targets built from critic action gradients, a soft pull toward the same-noise base action, and clipping of the combined displacement. Targets remain fixed during each actor update. Action-space operations use normalized coordinates ($D = Hd$), and the frozen base supplies samples without requiring policy likelihoods or gradients through its parameters. Fig. 2 illustrates the four-step actor update: sample, propose, constrain, and fit.

A. Sample Actions and Propose Corrections

Step 1: sample paired base and current actions. For each observation s from replayed demonstrations or policy rollouts, draw K independent noise samples $z_i \sim \mathcal{N}(0, I)$. Using the frozen pretrained policy π_0 , generate the base action $a_{0,i} = \pi_0(s, z_i)$ and a fresh candidate:

$$\tilde{r}_i = \text{sg}[r_\theta(s, z_i)], \quad a_i = a_{0,i} + \tilde{r}_i, \quad (2)$$

where sg stops gradients during target construction. Each paired base action provides a fixed restoration reference for the same (s, z_i) .

Step 2: ask the critic for a local correction. Let $\bar{Q}_\phi(s, z, a)$ denote the conservative critic-ensemble aggregate; we omit z for noise-independent critics. We divide its action gradient by the detached scale $q_{\text{scale}} = \max(\epsilon, \mathbb{E}_{\text{batch}}[\|\bar{Q}_\phi\|])$, computed over the current candidates, where $\epsilon > 0$ prevents division by zero. This reduces sensitivity to the overall magnitude of predicted values. Holding critic parameters

fixed, the proposed critic displacement is

$$d_i^Q = \eta_Q C_{\delta_Q} \left(\frac{\nabla_a \bar{Q}_\phi(s, z_i, a_i)}{q_{\text{scale}}} \right), \quad (3)$$

where $C_\delta(v) = v \min(1, \delta/\|v\|_2)$ and $C_\delta(0) = 0$. The scale η_Q sets the strength of critic guidance. The field cap δ_Q clips the normalized gradient *before* applying η_Q , so $\|d_i^Q\|_2 \leq \eta_Q \delta_Q$. The critic thus proposes a local change aimed at increasing predicted return. The next step balances this proposal against departure from the base policy.

B. Restore Toward the Base and Limit the Step

Step 3: combine improvement with a persistent reference. The detached residual $\tilde{r}_i = a_i - a_{0,i}$ measures the current candidate’s displacement from its paired base action. We express this departure as the root mean square (RMS) magnitude $m_i = \|\tilde{r}_i\|_2 / \sqrt{D}$ and define the restoration displacement as

$$d_i^R = -\eta_0 \tilde{r}_i - \eta_{\text{anc}} \left(1 - \frac{\rho}{\max(m_i, \epsilon)} \right)_+ \tilde{r}_i, \quad (4)$$

where $(x)_+ = \max(x, 0)$. The coefficient η_0 controls a weak pull toward the base at every nonzero residual, while η_{anc} controls an additional pull beyond the RMS radius ρ . This radius leaves room for adaptation under the weak pull before stronger restoration activates. It is a soft threshold, so targets may still lie outside it. Using RMS units makes ρ comparable across action-chunk dimensions.

We combine both displacements at the current candidate: critic guidance favors higher predicted return, and restoration resists accumulated departure from the base. We then clip their sum to construct the residual target:

$$\tilde{r}_i = \text{sg} \left[\tilde{r}_i + C_{\delta_{\text{tot}}} (d_i^Q + d_i^R) \right]. \quad (5)$$

The cap δ_{tot} sets the maximum requested change, ensuring $\|\tilde{r}_i - \tilde{r}_i\|_2 \leq \delta_{\text{tot}}$ even when restoration is large. The target is held fixed during the actor update. Together, restoration and clipping balance adaptation with a persistent reference by acting on accumulated departure and individual corrections, respectively. The component study in Sec. V-C tests this distinction.

C. Learn from Fixed Targets

Step 4: fit the residual to the corrected actions. Since the base is frozen, fitting the corrected action $\tilde{a}_i = a_{0,i} + \tilde{r}_i$ is equivalent to regressing the residual:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_s \left[\frac{1}{KD} \sum_{i=1}^K \|r_\theta(s, z_i) - \tilde{r}_i\|_2^2 \right], \quad (6)$$

where K is the number of sampled candidates per observation and $D = Hd$ is the action-chunk dimension, comprising H controls of d dimensions each. This mean-squared error is the actor’s only training loss. Targets are held fixed during each update and recomputed for the next, so gradients do not pass through target construction.

Training and execution. Algorithm 1 summarizes online training, retaining the baseline’s replay mixture, warm-up, exploration, and multi-sample n -step TD critic updates. Only

Algorithm 1 CAST: online residual fine-tuning

Require: Frozen policy π_0 , demonstrations $\mathcal{D}$, candidates per state K

- 1: Initialize residual r_θ , critic Q_ϕ , and target parameters $\phi^- \leftarrow \phi$
- 2: Seed replay $\mathcal{R}$ with base-policy rollouts
- 3: **for** each online iteration **do**
- 4: Collect policy rollouts and append them to $\mathcal{R}$
- 5: **for** each gradient step **do**
- 6: Sample a minibatch from $\mathcal{R}$ and $\mathcal{D}$
- 7: Compute n -step TD loss $\mathcal{L}_Q$ using Q_ϕ^-
- 8: **if** an actor update is due **then**
- 9: **Sample:** Form K paired candidates per state (2)
- 10: **Propose:** Compute critic corrections d_i^Q (3)
- 11: **Constrain:** Apply restoration and clipping to obtain fixed targets $\tilde{r}_i$ (4)–(5)
- 12: **Fit:** Regress r_θ onto $\tilde{r}_i$ (6)
- 13: Update ϕ using $\mathcal{L}_Q$
- 14: Update ϕ^- by Polyak averaging when due
- 15: **return** $\pi_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z)$

the critic uses a slowly updated target network. Physical refinement (Sec. V-B) trains offline on saved experience, with residual corrections limited to the portion of each chunk designated for execution. At inference, we sample candidate actions as $\pi_0(s, z) + r_\theta(s, z)$ using the base policy’s sampling procedure and apply each benchmark’s selection rule, including value-guided selection in the matched LIBERO comparisons.

V. EXPERIMENTS

We evaluate whether CAST improves pretrained skills and which update choices make refinement useful through three hypotheses:

- **H1 (useful refinement):** CAST improves task performance or reaches a target success rate with fewer environment steps than standard residual refinement.
- **H2 (a persistent reference):** Restoration toward the frozen base helps preserve task performance beyond limiting individual corrections.
- **H3 (preservation and adaptation):** The strength and form of preservation affect refinement; stronger constraints need not yield better performance.

Our evaluation spans state-based robomimic manipulation [28], image-based LIBERO-90 manipulation [29], dense- and sparse-reward DMC control [30], and physical OpenArm placement and stacking. Robomimic and LIBERO use sparse task-success rewards; DMC broadens the reward settings and task family. The physical study evaluates offline refinement from recorded experience followed by autonomous robot trials. Together, these settings test different observation modalities, reward structures, and deployment conditions. Simulation and physical comparisons test H1, the component comparison tests H2, and alternative constraints and restoration-radius sweeps test H3. DMC extends the evaluation to dense- and sparse-reward control.

Our primary comparison uses DICE-RL’s frozen-base residual framework and critic-training procedure. DICE-RL optimizes critic value with selectively applied BC regularization; CAST learns constrained action targets. Matched simulation comparisons share the pretrained checkpoint, residual

architecture, replay recipe, and budget, although collected experience and learned critics can diverge. We therefore compare the complete actor updates. Success rates are percentages and gains are percentage points (pp); additional protocol details are in the experimental supplement.

A. Policy Improvement in Simulation

Improvement across visual tasks. To test H1 on visual manipulation, we first train each multitask base on all 90 LIBERO tasks, then select eight challenging tasks for refinement. Each selected task receives its own residual policy and critic over the shared frozen base, with task-ID routing at deployment. Both base classes use the same eight-task subset. Table I reports two fixed values of the actor’s critic-guidance scale, η_Q , each shared across all tasks. On the drifting base, CAST₂ raises mean success from 66.1% to 75.5% (+9.4 pp), compared with 67.0% for DICE-RL (+0.9 pp). Both CAST settings improve over every task’s base. The benefit includes avoiding degradation: on red mug, DICE-RL falls below the base, whereas both CAST settings improve it substantially. DICE-RL still leads on Study-3 caddy.

The flow-matching base makes the dependence on the starting policy and update setting clear. Its mean success is 75.3%; CAST₁ reaches 75.6% and CAST₂ reaches 77.7%, compared with 75.0% for DICE-RL. The gain is smaller, and neither CAST setting improves every task. For example, all refinements fall below the strong juice-placement base, while doubling η_Q helps front-caddy placement but hurts right-caddy placement. The higher mean success over the eight selected tasks supports H1 across both base classes, while the task-level results show how the guidance scale affects individual skills.

Refinement across generator classes. Table II complements the visual study with state-based robomimic refinement on drifting, diffusion, and flow-matching bases. On drifting-base square, CAST improves a 38.2% base to 91.2–93.0%, exceeding DICE-RL on each of three training seeds at 20K updates. Substantial improvement is therefore compatible with a restoring reference to an imperfect base. CAST also improves can from a stronger starting policy, and its value-only variant improves flow-matching square from 57.0% to 93.4%. These gains support H1 across generator classes. The diffusion comparison below tests improvement speed separately; additional protocol details and external baseline comparisons are in the supplement.

Environment steps to useful performance. A higher final success rate and earlier improvement are distinct benefits. On the shared diffusion base for square, CAST first reaches 90% success at a mean of 149.3K environment steps, compared with 240K for DICE-RL (Fig. 3). This is approximately 38% fewer steps. At the common 496K endpoint, success is similar: 95.9% for CAST and 96.5% for DICE-RL. Reaching the success threshold with fewer environment steps supports H1’s efficiency claim.

B. Real-Robot Refinement on Stair-Based Manipulation

We evaluate a pretrained flow-matching policy on the OpenArm left arm and parallel gripper, using chest and wrist

TABLE I: LIBERO-90 success (%) on eight challenging tasks selected for refinement. Each base is pretrained on all 90 tasks and frozen; each selected task receives a separate residual policy and critic. CAST₁ and CAST₂ use critic-guidance scales $\eta_0 = 1$ and 2, respectively, fixed across tasks. Each result uses 100 rollouts per evaluation seed (three seeds, not independent training runs). Means cover only the eight selected tasks. Parentheses give task IDs; bold marks the highest result in each row and base block.

Task	Drifting base				Flow-matching base			
	Base	DICE-RL	CAST ₁	CAST ₂	Base	DICE-RL	CAST ₁	CAST ₂
Drawer / bowl (8)	81.8	82.0	83.1	85.7	89.7	87.3	90.0	88.7
Stove / pan (21)	69.0	70.3	73.7	75.2	70.0	73.0	69.0	72.7
Drawer / ketchup (32)	61.0	58.7	69.4	72.9	69.0	68.2	73.8	77.3
Basket / juice (53)	78.5	80.0	84.2	85.3	91.7	91.5	89.7	91.3
Plate / red mug (65)	57.3	55.0	82.6	81.2	78.7	80.9	87.8	88.7
Front caddy / book (73)	64.8	66.3	71.0	75.2	68.3	69.0	66.7	78.3
Right caddy / book (75)	57.8	58.3	65.7	64.8	62.0	59.2	60.8	56.3
Study-3 caddy / book (81)	58.8	65.3	60.7	63.5	73.3	70.8	67.0	68.0
Mean	66.1	67.0	73.8	75.5	75.3	75.0	75.6	77.7

TABLE II: Robomimic success (%) across pretrained generator classes. DICE-RL and CAST refine the same base in each row. Drifting ranges span three training seeds, each averaged over its last three 300-episode evaluations; † marks two DICE-RL can runs ending before 20K updates. Diffusion results use 496K environment steps. z: value-only CAST with an unspecified interaction budget. Bold marks the higher refinement result in each row.

Task / base class	Base	DICE-RL	CAST
Can / drifting	87.7	94.0–97.0 [†]	97.8–99.3
Square / drifting	38.2	86.1–90.9	91.2–93.0
Square / diffusion	60.0	96.5	95.9
Square / flow matching	57.0	83.1	93.4^z

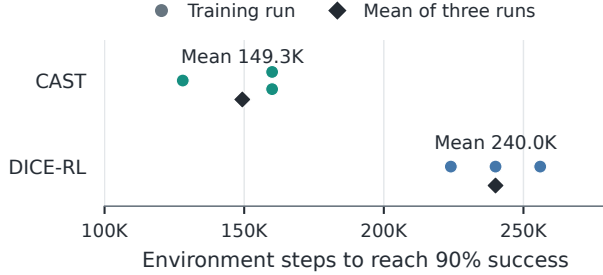


Fig. 3: CAST reaches 90% success sooner. On robomimic square, circles mark the first evaluated checkpoint reaching 90% success in each of three training runs from a shared diffusion base; diamonds mark the means. CAST requires 38% fewer steps on average. The two CAST runs reaching the threshold at 160K steps are offset vertically for visibility. Final success at 496K steps is similar: CAST 95.9%, DICE-RL 96.5%.

RGB observations together with joint and gripper state. The tasks are to place Jigglypuff into an adjacent case and to stack a red cube on a purple cube (Fig. 4). Both use a staircase: the stepped surfaces introduce different approach heights and nearby edges, making grasping and placement sensitive to alignment. Stacking additionally requires precise positioning and release without disturbing the supporting cube. This task design exposes a practical refinement problem: useful reaching, grasping, and transport can still end in an unsuccessful placement or release.

We compare the pretrained **Base**, supervised **BC** refine-

ment, **DICE-RL** with its original actor objective, and **CAST** with constrained action targets. Policy pretraining uses 40 placement demonstrations and approximately 60 stacking demonstrations. For each task, BC refinement uses 30 additional expert demonstrations. The two RL methods share 40 recorded robot rollouts per task, retaining both successful and failed attempts. Refinement then trains offline on the saved data. BC fine-tunes the base for 40 additional epochs. Each RL run learns a residual with the base and encoder frozen, using 6,000 critic updates. DICE-RL and CAST share the base, replay data and minibatch schedule, critic-training procedure, and update budget. BC therefore differs from RL in both supervision and trainable parameters.

After refinement, we evaluate each method in 20 autonomous attempts per task. Placement poses are randomized on the stairs, and success requires the toy to remain fully inside the case. Stacking succeeds when the red cube remains on the purple cube after release and gripper withdrawal. Detailed protocols are in the experimental supplement.

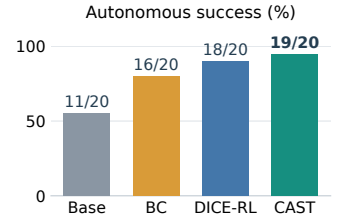
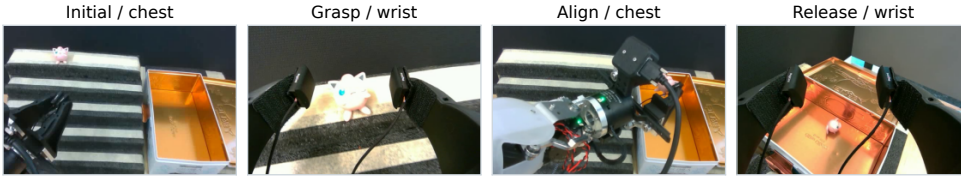
CAST raises placement success from 55% to 95% and stacking success from 35% to 70%, gains of 40 and 35 percentage points over the respective bases. These gains support H1 under offline refinement and physical deployment. Fig. 4 reports all four methods and their successful attempt counts; CAST achieves the highest observed success on both tasks.

The main observed stacking failure is failure to release the gripper after useful reaching, grasping, and alignment. Refinement should retain those useful actions while correcting the release. BC receives explicit release targets from expert demonstrations; CAST moderates critic-guided changes through restoration and clipping. The observed ordering is consistent with preserving useful actions while correcting task-ending mistakes.

C. The Role of Restoration and Clipping

Is limiting each correction sufficient, or must refinement also respond to accumulated departure from the base? The five-task LIBERO component comparison in Fig. 5 evaluates no controls, clipping only, radial restoration only, and full CAST. The first two remove both restoring terms. Radial restoration alone also sets $\eta_0 = 0$, whereas full CAST enables

(a) Jigglypuff into the case



(b) Red cube onto the purple cube

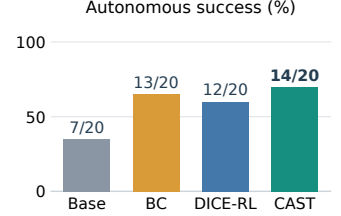
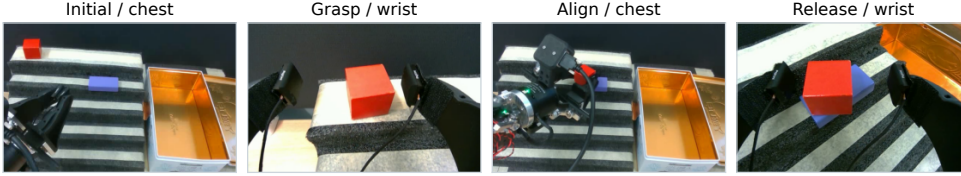


Fig. 4: Higher autonomous success on physical OpenArm tasks. Rows show placement (top) and stacking (bottom) on a staircase. Images depict successful *teleoperated demonstrations*, with chest and wrist views of grasping, alignment, and release. Bars report *separate autonomous evaluations* on a common 0–100% scale; labels give successful attempts out of 20, with the highest count in bold. BC, DICE-RL, and CAST refine the same pretrained flow-matching base offline.

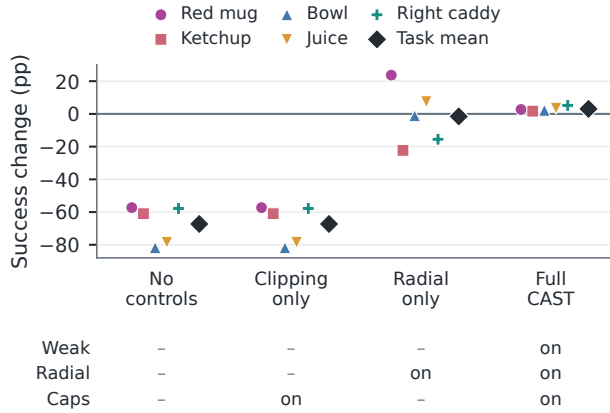


Fig. 5: Restoration preserves performance beyond clipping alone. Colored points show success changes in percentage points from five LIBERO tasks’ respective bases; diamonds mark the task mean. Points are task aggregates, not training seeds. Both configurations without restoration have zero success on all five tasks. The key shows weak and radial restoration and the action-space caps. Radial-only restoration also omits the weak pull, so its comparison with full CAST does not isolate clipping.

the weak pull, radial restoration, and both action-space caps. This is a component comparison: the last contrast changes both clipping and the weak restoring term.

Both configurations without restoration reach zero reported success on all five tasks, even when each requested correction is capped. Restoration variants retain nonzero success throughout. This supports H2 in the tested setting: a cap on individual corrections does not replace a persistent reference. The distinction follows the update’s design: clipping limits the current request, while restoration responds to the departure already accumulated from the frozen base.

Retaining some success does not ensure improvement. Radial restoration alone averages 65.7%, below the base’s 67.3%, and improves only two tasks. Full CAST reaches 70.3% and improves every task over its base, although radial restoration alone remains better on red mug and juice.

TABLE III: Success (%) with alternative constraints and critic guidance. Both studies use the 38.2% drifting base on robomimic square; mug and ketchup are LIBERO tasks 65 and 32. Constraint results in (a) are reported over three training seeds; square results in (b) are ranges across three training seeds. The studies use separate CAST references and auxiliary settings (see supplement). Bold marks the highest result per task within each block.

(a) Constraint implementations			
Constraint	Square	Mug	Ketchup
Base	38.2	57.3	61.0
Loss-space hinge	48.0	77.0	65.0
Hard target projection	96.0	93.0	54.0
Full CAST	97.0	78.0	68.0

(b) Critic-guidance recipes			
Guidance	Square	Mug	Ketchup
Base	38.2	57.3	61.0
Gradient	91.2–93.0	60.0	62.7
Top- k	52.5–56.2	57.7	34.1
Value-tilted	56.0–58.4	69.7	37.8

Together, these results support the full update as a useful balance between retaining pretrained behavior and improving task performance.

D. Choosing the Constraint and Critic Guidance

Does harder preservation help? To test H3, Table III(a) compares a penalty in the actor loss, a hard constraint on the target, and CAST’s soft restoration. The *loss-space hinge* uses $(m - \rho)_+^2$ as DICE-RL’s residual penalty, where $m = \|r_\theta(s, z)\|_2 / \sqrt{D}$ is the RMS departure from the base, while retaining its BC filtering. *Hard target projection* rescales the residual target to satisfy $\|\tilde{r}_i\|_2 \leq \sqrt{D}\rho$, a ball centered on the base action. CAST instead restores toward the base while capping the target’s *step from the current action* in (5). The LIBERO projection recipe replaces restoration and both caps; auxiliary settings for these update recipes are given in the supplement.

CAST reaches 97.0% on square, compared with 96.0% for projection, and leads on ketchup at 68.0%. Projection

remains best on mug (93.0% versus 78.0% for CAST), but falls below the ketchup base. The hinge improves square from 38.2% to 48.0%, but remains well below projection and CAST. This task-dependent ordering supports H3: stronger preservation does not uniformly improve refinement. Restricting departure can retain useful behavior while also limiting useful changes; the observed reversals are consistent with this tradeoff.

Can value-only guidance replace action gradients? Table III(b) tests whether refinement can use the critic’s scores without differentiating them with respect to actions. Gradient guidance proposes a local correction; the two value-only variants instead use kernel-weighted attraction and repulsion. *Top-k* attracts candidates toward the k highest-valued current samples. *Value-tilted* guidance weights all sampled attractors proportionally to $\exp(\hat{q}_i/\tau)$, where $\hat{q}_i$ is the critic value standardized within the state’s candidate set and $\tau > 0$ is a temperature. Both repel from uniformly weighted current samples. Their weak restoration uses independent base samples in place of the paired pull $-\eta_0 \bar{r}_i$; radial restoration and displacement caps are retained with the settings reported in the supplement.

The gradient recipe performs best on square and ketchup. Both value-only recipes improve over the square base but fall below the ketchup base. On mug, value-tilted transport exceeds the gradient recipe. These results show that value-only guidance can improve a pretrained policy, while the relative benefit of action gradients depends on the task. This connects H1’s refinement gains to the choice of critic guidance.

Allowing room before stronger restoration. The radius ρ determines when the stronger restoring term activates. At $\rho = 0$, it acts on every nonzero residual. In exploratory single-training-seed sweeps on mug and ketchup, positive radii from 0.02 to 0.20 improve success over this setting. For example, $\rho = 0.05$ raises mug success from 54.9% to 78.1% and ketchup success from 60.1% to 68.4%. This provides exploratory support for H3: allowing local adaptation before stronger restoration can improve refinement. The supplement reports every radius and result. As $\rho \rightarrow \infty$, the weak pull remains, so that setting still differs from removing restoration. Together, these studies show why both the proposed correction and the way it is moderated matter.

E. Refinement under Dense and Sparse Rewards

Table IV reports DMC refinement of one-step bases distilled from SAC experience under dense and sparse rewards. With dense rewards, CAST improves swing-up from 429 to 715 return and balance from 378 to 911. With sparse rewards, it improves balance from 90 to 1000 and ball-in-cup catch from 86 to 925. These gains extend the evidence for H1 beyond manipulation to both reward settings. Unconstrained refinement also improves the two sparse tasks, showing that the observed role of restoration varies across control settings. The table caption and supplement provide evaluation and training-budget details.

In summary, Tables I–II and Figs. 3–4 support H1 through

TABLE IV: Latest DMC returns (0–1000) under dense and sparse rewards. Methods share the same base per task. Each return averages ten 1000-step evaluation episodes; swing-up uses two training seeds and other tasks use one. Some runs end before the 400K environment-step target (see supplement). v : high evaluation variance; t : tuned actor guidance ($\eta_Q = 1$). Bold marks the highest reported return, including interim results and ties.

Task	Base	Unconstrained	DICE-RL	CAST
<i>Dense reward</i>				
Cartpole swing-up	429	575	318	715
Cartpole balance	378	815	844	911
Quadruped run	238	97	283	312'
Finger spin	3	0	187	187
<i>Sparse reward</i>				
Cartpole balance	90	880	843 ^v	1000
Ball-in-cup catch	86	940	66	925

higher success or earlier improvement. Fig. 5 supports H2 by showing that clipping alone does not preserve performance. Table III(a) and the radius sweep support H3: stronger preservation does not uniformly help. The appropriate balance depends on the refinement setting.

VI. DISCUSSION AND CONCLUSION

We presented CAST, a likelihood-free method for refining pretrained generative robot policies through residual learning. CAST combines critic guidance with soft restoration toward the frozen base and clips each proposed correction to construct fixed action targets. The residual then learns these targets by regression. Experiments with state-based and visual manipulation policies show improvements in task success and fewer interactions needed to reach a target success rate. Dense- and sparse-reward control tasks provide further evidence of useful refinement, while offline OpenArm experiments demonstrate gains in physical placement and stacking.

The central design lesson is to maintain a reference to useful behavior while allowing the changes needed for improvement. In the five-task LIBERO component study, restoration preserves task success where clipping alone fails. The constraint and guidance comparisons also show that stronger preservation is not uniformly better: its value depends on the task and the proposed correction. This balance is relevant to robot skills that already reach and grasp successfully but still need to improve placement or release. Adapting restoration to local critic reliability and expanding physical evaluations are the next steps toward retaining these useful behaviors while learning more reliable task completion.

REFERENCES

- [1] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy optimization,” in *Proc. ICLR*, 2025, arXiv:2409.00588.
- [2] T. Zhang, C. Yu, S. Su, and Y. Wang, “ReinFlow: Fine-tuning flow matching policy with online reinforcement learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2505.22094>
- [3] Z. Sun and S. Song, “From prior to pro: Efficient skill mastery via distribution contractive RL finetuning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10263>
- [4] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proc. ICML*, 2018.

- [5] S. Fujimoto, D. Meger, and D. Precup, “Off-policy deep reinforcement learning without exploration,” in *Proc. ICML*, 2019.
- [6] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin, “Policy representation via diffusion probability model for reinforcement learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.13122>
- [7] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a diffusion model policy from rewards via Q-score matching,” in *Proc. ICML*, 2024, pp. 41 163–41 182.
- [8] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, “Flow matching policy gradients,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.21053>
- [9] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion policies as an expressive policy class for offline reinforcement learning,” in *Proc. ICLR*, 2023.
- [10] S. Park, Q. Li, and S. Levine, “Flow Q-learning,” in *Proc. ICML*, 2025, pp. 48 104–48 127.
- [11] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” in *Proc. CoRL*, 2025, pp. 258–282.
- [12] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering your generalists: Improving robotic foundation models via value guidance,” in *Proc. CoRL*, 2025, pp. 4996–5013.
- [13] H. Hu, S. Mirchandani, and D. Sadigh, “Imitation bootstrapped reinforcement learning,” in *Proc. RSS*, 2024.
- [14] M. S. Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar, “Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.06685>
- [15] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.
- [16] A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller, “Maximum a posteriori policy optimisation,” in *Proc. ICLR*, 2018.
- [17] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, “IDQL: Implicit Q-learning as an actor-critic method with diffusion policies,” *arXiv preprint arXiv:2304.10573*, 2023.
- [18] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, “Reinforcement learning with deep energy-based policies,” in *Proc. ICML*, 2017.
- [19] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.04770>
- [20] J. Koo, M. Park, J. Choi, Y. Min, and M. Sung, “Drifting field policy: A one-step generative policy via wasserstein gradient flow,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.07727>
- [21] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. Aparicio Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *Proc. IEEE ICRA*, 2019, pp. 6023–6029.
- [22] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, “Residual off-policy RL for finetuning behavior cloning policies,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.19301>
- [23] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, “Policy decorator: Model-agnostic online refinement for large policy model,” in *Proc. ICLR*, 2025.
- [24] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, “What makes value learning efficient in residual reinforcement learning?” 2026. [Online]. Available: <https://arxiv.org/abs/2602.10539>
- [25] A. Kumar, J. Fu, M. Soh, G. Tucker, and S. Levine, “Stabilizing off-policy Q-learning via bootstrapping error reduction,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [26] S. Fujimoto and S. S. Gu, “A minimalist approach to offline reinforcement learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.
- [27] X. Cheng, H. Wang, W. Yuan, Z. Wang, Z. Chen, L. Zeng, and Z. Sun, “Fisher decorator: Refining flow policy via a local transport map,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.17919>
- [28] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proc. CoRL*, 2021.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [30] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq *et al.*, “DeepMind control suite,” *arXiv preprint arXiv:1801.00690*, 2018.